

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: «Редакционное расстояние »

Студент гр. 4345

Андросов М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Левенштейна

Задание

Над строкой ϵ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $replace(\epsilon, a, b)$ – заменить символ a на символ b .
2. $insert(\epsilon, a)$ – вставить в строку символ a (на любую позицию).
3. $delete(\epsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A ; третья строка – B .

Выходные данные: одно число – минимальная стоимость операций.

Sample Input:

1 1 1

entrance

reenterable

Sample Output:

5

Вариант 4р.

4а. Добавляется 4-я операция со своей стоимостью: замена одного символа на два символа.

Выполнение работы

Разработан алгоритм нахождения минимальной стоимости операций для превращения строки A в строку B.

Описание алгоритма

Строится матрица dp размером $n+1 * m+1$. Каждый элемент $dp[i][j]$ хранит минимальную стоимость преобразование префикса первой строки длины i в префикс второй строки длины j . Вычисляется динамически путем выбора минимума из каждой операции.

Оценка сложности алгоритма

Временная сложность: $O(N*M)$

Пространственная сложность $O(N*M)$

где N – длина 1 строки , а M – длина 2 строки.

Разработанный программный код см. в приложении А.

Тестирование

Таблица 1 — Результаты тестирования.

№	Входные данные	Выходные данные
1	1 1 1 1 abcd aaa	3
2	1 1 1 1 abc a	2
3	1 1 1 1 simular simular	0
4	1 1 1 1 longstory lonnnqag	5

Вывод

В ходе работы был выполнен алгоритм нахождения минимальной стоимости операций для превращения строки А в строку В.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.py

```
def minDistance(word1: str, word2: str, replace : int, insert: int, delete:
int, change: int) -> int:
    n = len(word1)
    m = len(word2)
    if m < n:
        n,m = m,n
        word1 ,word2 = word2,word1

    dp = [[0] * (m+1) for _ in range(n+1)]

    dp[0][0] = 0

    for i in range(1,n+1):
        dp[i][0] = delete + dp[i-1][0]

    for j in range(1,m+1):
        dp[0][j] = insert + dp[0][j-1]

    for i in range(1,n+1):
        for j in range(1,m+1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                changed = float("inf")
                if j >1: changed = change + dp[i-1][j-2]
                dp[i][j] = min(
                    changed,
                    replace + dp[i-1][j-1],
                    delete + dp[i-1][j],
                    insert + dp[i][j-1],
                )
    print_matrix(dp,word1,word2)
    return dp[n][m]
```

```

def print_matrix(dp, s1, s2):
    print("    ", " ".join(f"{c:>3}" for c in " " + s2))
    for i, row in enumerate(dp):
        char = " " if i == 0 else s1[i-1]
        print(f"{char:>3}", " ".join(f"{v:>3}" for v in row))

```

```

if __name__ == "__main__":
    replace, insert, delete, change = map(int, input().split())
    w1 = input()
    w2 = input()
    ans = minDistance(w1, w2, replace, insert, delete, change)
    print(ans)

```